

KERAS OUTPUT LAYER, ACTIVATION FUNCTIONS, LOSS FUNCTIONS & METRICS

Complete Teacher Notes

A classroom-ready guide to choosing the final layer, activation, loss function and evaluation metrics from the **problem definition**.

Core teaching chain

Problem type → **target structure** → **output representation** → **final activation** → **loss** → **metrics**

Teacher objective

Students should be able to look at a problem and explain **what the network must output, why that output needs a particular activation, why the loss matches the target, and what each metric actually tells us.**

Do not teach this as a memory table

The most reliable approach is to derive the answer from the target. Ask: **What can be true for one example?** That question separates binary, multiclass single-label and multiclass multi-label problems.

Contents

- 1. The big picture: prediction, loss and metrics
- 2. Identify the task from the target
- 3. Binary classification
- 4. Multiclass, single-label classification
- 5. Multiclass, multi-label classification
- 6. Regression
- 7. Why activation and loss must match
- 8. Metrics in practical terms
- 9. Keras implementation patterns
- 10. Classroom delivery plan
- 11. Common mistakes
- 12. One-page recap

1. The Big Picture: Prediction → Error → Improvement

A neural network does not simply “give an answer.” It produces numerical outputs. The **meaning** of those numbers depends on the problem.

Keep these three ideas separate:

- **Output:** What numbers should the final layer produce?
- **Loss:** How wrong is the prediction during training?
- **Metric:** How do we report model performance?

Teacher analogy

The output format is the student's answer, the loss is the marking scheme, and the metric is the scorecard. They work together but are not the same thing.

A useful neural-network training picture

**Input → hidden layers → final layer → prediction → loss → gradients → optimizer
update**

The final layer is therefore not an arbitrary design choice. It is the point where the network's internal numerical representation is converted into a form that matches the target.

Teacher checkpoint

Ask: “If I only tell you that we are using a neural network, can you choose the output activation?” **No.** First identify the target.

2. Identify the Task from the Target

Start with the label/target—not with the activation function.

Task	Question	Target	Output idea
Binary classification	Which of two outcomes?	0 / 1	One probability
Multiclass, single-label	Which one of C classes?	Exactly one class	C competing probabilities
Multiclass, multi-label	Which labels apply?	Several 0/1 labels	C independent probabilities
Regression	What numerical value?	Continuous number	Numeric value

The key distinction: single-label vs multi-label

Single-label: one example has exactly one final class. MNIST image → one digit.

Multi-label: several labels may be true simultaneously. Image → dog = 1, person = 1, car = 1.

Fast classroom test

Ask: “Can two labels be TRUE for the same example?” No → single-label multiclass. Yes → multi-label.

3. Binary Classification

Binary classification means there are two mutually exclusive outcomes, commonly encoded as 0 and 1.

Example: **spam detection**. 0 = ham, 1 = spam.

Input	True target	Model output	Decision example
Email features	1 = spam	0.93	Threshold 0.50 → spam

Why sigmoid?

Sigmoid maps a real-valued score to a number between 0 and 1, making it suitable for a binary probability output.

$$\sigma(z) = 1 / (1 + e^{-z})$$

A value near 1 indicates strong support for the positive class; a value near 0 indicates strong support for the negative class.

Why binary crossentropy?

Binary crossentropy compares the true binary target with the predicted probability and heavily penalizes confident wrong predictions.

$$L = -[y \log(p) + (1-y) \log(1-p)]$$

Example: true $y = 1$. Prediction $p = 0.95 \rightarrow$ small loss. Prediction $p = 0.05 \rightarrow$ large loss.

Keras implementation

```
model = keras.Sequential([
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy", keras.metrics.AUC(name="roc_auc")]
)
```

Explain the table in one sentence

Binary → one probability → sigmoid → binary crossentropy → binary accuracy / ROC AUC.

Metric interpretation

Binary accuracy: proportion of correct thresholded decisions. **ROC AUC:** how well the model separates/ranks positives above negatives across thresholds. Accuracy can be misleading with severe class imbalance.

4. Multiclass, Single-Label Classification

There are more than two possible classes, but every example belongs to exactly one class.

Example: **MNIST** has ten classes: digits 0 through 9.

Why softmax?

Softmax converts C final scores into a probability distribution across the C classes. The probabilities are non-negative and sum to 1.

$$\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

Class	0	1	2	3	4	...	9
Probability	0.01	0.00	0.02	0.91	0.01	...	0.01

The predicted class is normally the class with the highest probability. Here, the model predicts digit 3.

Why categorical crossentropy?

With one-hot targets, the correct class is 1 and all other classes are 0. Categorical crossentropy rewards assigning high probability to the correct class.

$$L = -\sum y_i \log(p_i)$$

Keras implementation — one-hot labels

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["categorical_accuracy"]
)
```

Important Keras nuance

Categorical crossentropy is appropriate for **one-hot encoded** targets. If targets are integer class IDs (0, 1, ..., 9), use **sparse_categorical_crossentropy**. The task itself has not changed.

Top-k accuracy

Top-3 accuracy asks: "Is the correct class among the three highest-probability classes?" This is useful when several likely candidates are valuable.

ROC AUC can be extended to multiclass classification using an appropriate multiclass strategy, but accuracy and top-k accuracy are generally easier first metrics for this classroom example.

5. Multiclass, Multi-Label Classification

There are multiple possible labels, and **several labels can be correct for one example**. Each output is therefore an independent binary decision.

Example: image tagging.

Image	True labels	Predicted probabilities
Street photograph	car = 1 person = 1 dog = 0 bicycle = 0	car = 0.92 person = 0.81 dog = 0.12 bicycle = 0.07

Notice: car and person can both be 1. The probabilities do **not** need to add to 1.

Why sigmoid?

Each output neuron independently answers “How likely is this label?” Hence C independent sigmoid outputs.

Dense(C, activation='sigmoid')

Why binary crossentropy?

Each label is a binary 0/1 prediction, so binary crossentropy is applied across the labels.

Keras implementation

```
model = keras.Sequential([
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(4, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["binary_accuracy"]
)
```

Critical contrast

Single-label multiclass: classes compete → softmax.

Multi-label: labels are independent → sigmoid.

Thresholds

A sigmoid value such as 0.81 does not automatically mean the label must be selected. A threshold is applied for a hard decision. 0.5 is common, but the appropriate threshold can depend on the application and the cost of false positives and false negatives.

6. Regression

Regression predicts a continuous numerical value rather than a category.

Examples: house price, sales amount, temperature, demand, delivery time.

Why no activation?

The final output generally needs to be a flexible numeric value. A linear output is therefore used. In Keras, **Dense(1)** with no activation means a linear output.

```
model = keras.Sequential([
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(1) # linear output
])
```

Why mean squared error (MSE)?

MSE calculates the average squared difference between actual and predicted values.

$$\text{MSE} = (1/n) \sum (y - \hat{y})^2$$

Squaring means large errors receive disproportionately more weight.

Why mean absolute error (MAE) as a metric?

MAE calculates the average absolute prediction error.

$$\text{MAE} = (1/n) \sum |y - \hat{y}|$$

MAE is easy to interpret because it uses the same units as the target. If house price is measured in rupees, MAE is in rupees.

Keras implementation

```
model = keras.Sequential([
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)
```

Teacher explanation

Loss: training objective. **Metric:** reporting/evaluation lens. MSE emphasizes large errors; MAE communicates typical error magnitude in target units.

7. Why Activation and Loss Must Match

The final activation determines the mathematical form and interpretation of the output. The loss must be compatible with that target/output relationship.

Task	Output	Activation	Loss
Binary	One probability	Sigmoid	Binary crossentropy
Multiclass single-label	C competing probabilities	Softmax	Categorical crossentropy*
Multiclass multi-label	C independent probabilities	Sigmoid	Binary crossentropy
Regression	Continuous number	Linear / none	MSE

Why not softmax for multi-label?

Softmax creates competition and forces the outputs to sum to 1. If an image contains both a dog and a car, forcing the labels to compete is not the desired representation.

Why not sigmoid for ordinary single-label multiclass?

Independent sigmoid outputs do not naturally encode “exactly one of these classes.” Softmax directly represents a probability distribution over mutually exclusive classes.

Why not MSE as the standard classification loss?

MSE can be used in some setups, but crossentropy is the standard probabilistic objective for these classification tasks and provides an appropriate training signal.

Memorization shortcut

Do not teach “multiple classes = softmax” without qualification. Teach “**multiple classes + exactly one correct class = softmax.**”

8. Metrics: What They Actually Tell Us

A metric is not merely a number to print at the end. It answers a particular evaluation question.

Metric	Question answered	Important property
Binary accuracy	How many thresholded decisions are correct?	Threshold-dependent
ROC AUC	How well are positives ranked above negatives?	Considers thresholds
Categorical accuracy	Is the highest-probability class correct?	Single-label multiclass
Top-k accuracy	Is the correct class among the top k?	Useful when several candidates matter
MAE	How far away are predictions on average?	Same units as target

Accuracy and imbalance

Suppose 990 transactions are legitimate and 10 are fraudulent. A model that predicts “legitimate” every time has 99% accuracy but detects no fraud. This demonstrates why accuracy should be interpreted in context.

ROC AUC

ROC AUC is useful for judging discrimination/ranking across classification thresholds. It is different from accuracy: accuracy evaluates a particular thresholded decision; AUC evaluates ranking behavior across thresholds.

Regression

MAE answers “How large is the typical absolute error?” MSE answers a similar question but squares errors, so large mistakes matter more.

Teacher question

“Can a model have high accuracy but still be a bad fraud detector?” Yes. Ask students to explain why. This creates the bridge from metrics to business context.

9. Keras Implementation Patterns

Use these patterns as the practical reference during the class.

A. Binary classification

```
Dense(1, activation="sigmoid")
loss="binary_crossentropy"
metrics=["accuracy"]
```

B. Multiclass single-label — one-hot targets

```
Dense(C, activation="softmax")
loss="categorical_crossentropy"
metrics=["categorical_accuracy"]
```

C. Multiclass single-label — integer targets

```
Dense(C, activation="softmax")
loss="sparse_categorical_crossentropy"
metrics=["sparse_categorical_accuracy"]
```

D. Multiclass multi-label

```
Dense(C, activation="sigmoid")
loss="binary_crossentropy"
metrics=["binary_accuracy"]
```

E. Regression

```
Dense(1)                # linear output
loss="mse"
metrics=["mae"]
```

Output-neuron rule

Binary → **1** sigmoid neuron. Single-label C-class → **C** softmax neurons. Multi-label C-label → **C** sigmoid neurons.
One-target regression → **1** linear neuron.

10. Classroom Delivery Plan

A strong teaching sequence prevents the table from becoming a memorization exercise.

Step 1 — Give the real-world problem

- Spam or not spam? → binary
- Which digit 0–9? → multiclass single-label
- Which objects are present? → multiclass multi-label
- What will the house sell for? → regression

Step 2 — Ask what the network must output

- One probability
- C probabilities that compete and sum to 1
- C independent probabilities
- One continuous number

Step 3 — Choose the activation

- One probability → sigmoid
- Competing class probabilities → softmax
- Independent label probabilities → sigmoid
- Continuous numeric output → linear/no activation

Step 4 — Choose the loss

- Binary probability → binary crossentropy
- One-hot multiclass → categorical crossentropy
- Independent binary labels → binary crossentropy
- Continuous value → MSE

Step 5 — Choose metrics

Only after students understand the prediction should you ask how performance should be measured.

Teacher questioning script

Ask	Expected reasoning
What is the target?	Category or continuous number?
How many classes?	Two or C classes?
Can multiple labels be true?	No → single-label; yes → multi-label
What should the output mean?	Probability/probability distribution/independent probabilities/numeric value
Which activation?	Sigmoid / softmax / sigmoid / linear
Which loss?	BCE / categorical CE / BCE / MSE

11. Common Mistakes and Troubleshooting

1. “Multiclass always means softmax.”

Only when the multiclass problem is **single-label**. Multi-label classification normally uses independent sigmoid outputs.

2. “Sigmoid means binary only.”

Sigmoid is also used independently for each label in multi-label classification.

3. Categorical vs sparse categorical crossentropy

Categorical crossentropy is typically paired with one-hot encoded targets. Sparse categorical crossentropy is typically paired with integer class IDs.

4. Loss and metric are the same thing

They are not. Loss drives the training objective; metrics are evaluation/reporting measures.

5. Threshold 0.5 is a universal law

It is a common default for sigmoid classification, but the best threshold depends on the application's costs and desired trade-off.

6. ROC AUC is “another accuracy”

No. AUC assesses ranking/discrimination across thresholds, while accuracy assesses correctness at a selected threshold.

7. Softmax on regression

Softmax creates a probability distribution that sums to 1. Regression generally needs a linear numeric output.

8. Wrong target shape/encoding

When Keras raises output/loss shape errors, inspect the final layer, the target shape, and whether labels are one-hot or integer encoded.

Diagnostic checklist

Final Dense layer → target shape → target encoding → activation → loss → metric. Many “mysterious” Keras errors are actually mismatches between these pieces.

12. One-Page Recap

Task	Example	Last-layer activation	Loss	Metrics
Binary classification	Spam detection	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
Multiclass, single-label	MNIST	Softmax	Categorical crossentropy*	Categorical accuracy, top-k, ROC AUC*
Multiclass, multi-label	Image tagging	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
Regression	House price	None / linear	Mean squared error	Mean absolute error

* Categorical crossentropy assumes one-hot targets; with integer class IDs, use sparse categorical crossentropy. Multiclass ROC AUC requires an appropriate multiclass formulation.

The one-minute mental model

Remember this

Task determines the output. Output determines the activation. Target representation determines the loss. The evaluation question determines the metric.

Board summary

BINARY

0/1 → one probability → SIGMOID → BINARY CROSSENTROPY
→ binary accuracy / ROC AUC

MULTICLASS SINGLE-LABEL

one of C → C competing probabilities → SOFTMAX → CATEGORICAL CROSSENTROPY
→ accuracy / top-k

MULTICLASS MULTI-LABEL

several of C → C independent probabilities → SIGMOID → BINARY CROSSENTROPY
→ binary accuracy / ROC AUC

REGRESSION

continuous value → numeric output → LINEAR/NONE → MSE
→ MAE

Suggested final question to the class

“You receive a new problem tomorrow. Without knowing the dataset, what four things do you inspect before writing the final Dense layer?” Expected: **target type, number of classes/labels, whether labels can co-exist, and target encoding.**

End of complete teacher notes.